Segmenting Solar Filaments When the Annotators Disagree: One Network and Two Keep-Scorers

A Solution to the Solar Filament Segmentation Challenge 2026

Muhammad Elias Muhammadi*
Independent Researcher
Kabul, Afghanistan
Muhammad.Elias.Muhammadi@gmail.com

ABSTRACT

This report describes our solution to the Solar Filament Segmentation Challenge 2026, a Kaggle competition on automatic segmentation of solar filaments in GONG H-α observations. I first measured what the labels allow: two annotators of the same frame agree with a Panoptic Quality (PQ) of 0.34, and the mean of their masks scores 0.78 against them. The pipeline that follows from this is small. One ConvNeXt-T U-Net predicts, at native resolution, the probability that an annotator marks a pixel; connected components of the thresholded map are the candidates; a boosted-tree scorer and a small CNN on candidate patches estimate the IoU each candidate will reach, and a candidate is kept when the estimate exceeds half the expected PQ. The out-of-fold PQ on the 707 training frames is 0.459 (90% interval 0.451 to 0.467) at 0.35 s per frame. I did not use the test annotations that are public in the MAGFiLO release, and I report the ideas that did not help. Code, weights and a notebook that reproduces the submission are at https://github.com/Muhammad-Elias-Muhammadi/Solar-Filament-Segmentation.

1 INTRODUCTION

This report goes over the details of our submitted solution to the Solar Filament Segmentation Challenge 2026 [2]. The challenge, launched on July 10, 2026, is organized by the Earth-Space AI Research (ESAIR) Lab[1]. This challenge utilizes the MAGFiLO dataset for evaluation of the segmentation algorithms [1]. The training set has 707 full-disk frames from the six GONG sites [8] with 1,154 annotator records and 8,199 filament polygons; 411 frames were drawn by one annotator, 145 by two and 151 by three. A submission is one set of disjoint masks per frame, scored against each record of the frame with PQ [12] at IoU > 0.5, the counts pooled.

Before building anything larger I scored the labels against themselves with the same metric (Table 1). Three things follow. First, the annotators disagree far more than the models differ: one annotator against another reaches 0.34, and so does the union or intersection of two against the third. The disagreement is about which faint filaments exist and where a filament ends (Figure 2b). Second, a submission cannot satisfy two or three annotators at once, so even the mean of the annotators' own masks stops at 0.78. Third, grouping pixels into instances is not the difficulty: the connected components of an annotator's own mask reproduce that annotator's filaments almost exactly, because only 30 of the 8,199 filaments are drawn in several pieces and 10 touch a neighbour. I therefore dropped the instance detector of my earlier versions and treated the problem as two questions: what is the probability that an annotator marks a pixel, and which components of that map are worth keeping.

Table 1: The labels scored against themselves and against my models (PQ, training frames).

scored against the annotators	PQ
another annotator of the same frame	0.340
union / intersection of two, against the third	0.334 / 0.318
this work, out-of-fold	0.459
this work with perfect keep decisions	0.519
mean of the annotators' masks, all frames	0.745–0.782
frames with one annotator	1.00
frames with two / three annotators	0.55–0.71 / 0.56–0.61
components of an annotator's own mask	0.997

Related work. Deep networks have replaced thresholding and region growing for filament segmentation in Hα images: U-Nets and their variants [13, 18], detectors that also classify filaments [7], and a semi-supervised U-Net that transfers across observatories [4]. A Mask R-CNN on full-disk frames was an early instance-level approach on GONG data [3]. These works train against one mask per image and mostly report pixel scores. Here a frame can carry up to three masks that disagree, and the score is instance-level and pooled over the annotators. I therefore treat the annotators as a population to be modelled, in the spirit of learning from individual labelers [6], and choose the output that maximises the expected metric.

Contributions. (1) A measurement of what the labels allow under the competition metric (Table 1). (2) A keep rule derived from the metric itself: a candidate is kept exactly when its expected matched IoU exceeds half the PQ. (3) A patch-based CNN scorer which, averaged with a feature scorer, is the only change to the decoder that helped. (4) Ten ideas that did not help, each tested through the complete decoder (Table 3).

*ORCID: https://orcid.org/0009-0006-7490-0476. Implementation (code, weights and notebook): https://github.com/Muhammad-Elias-Muhammadi/Solar-Filament-Segmentation.
[1]www.esairlab.com/

Kaggle Competition, 2026, kaggle.com/competitions/filament-segmentation-2026
2026.

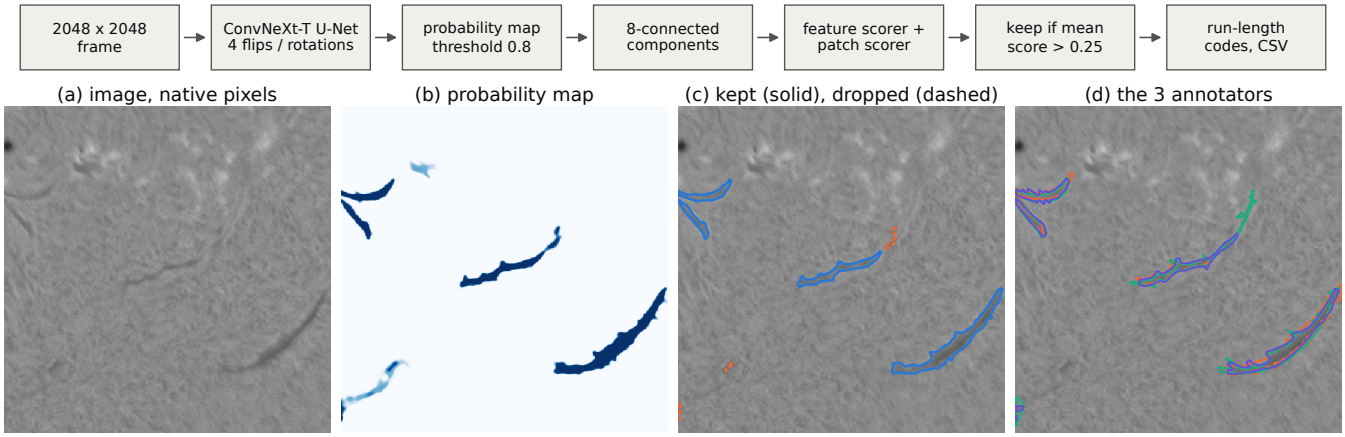


Figure 1: The pipeline (top row) and one 400×400 px region of a validation frame going through it. The network turns the image (a) into a probability map (b). Components above 0.8 are candidates; the two scorers keep the filaments and drop the small pieces (c). The three annotators of this frame (d, one colour each) agree on the cores and differ at the ends.

2 METHODOLOGY

Figure 1 shows the whole pipeline. It uses existing parts: a U-Net [10, 16] with a ConvNeXt-T encoder [14, 17], connected components, a gradient-boosted regressor [15] and a ResNet-18 [9]. What is specific to this task is the target, the keep rule and the second scorer.

Data, target and splits. I use only the Kaggle bundle. There is no pre-processing beyond reading the 8-bit frame; nothing is downsampled, because a filament is often three to six pixels wide and at half resolution the IoU of such a ribbon is decided by rounding. The target of a frame is the mean of its annotators' masks, rasterised at 2048×2048, so a pixel marked by one of three annotators has the value $1/3$. I use five folds for training and validation; the test set is the one of the competition. The folds are built over frames, never over records; whole 27-day blocks go to one fold because a filament lives for days, and the folds are stratified by site.

Network and training. The U-Net has 31.9 M parameters, one grey input channel and two outputs: the filament probability and an auxiliary centre line. Each step draws twelve 1024×1024 crops at native scale. The augmentation follows what varies between GONG frames: zoom by 0.85–1.25, any rotation and flip, gamma, contrast and brightness, Gaussian blur and a down-and-up resampling for seeing, a smooth multiplicative field for thin cloud, and a change of the limb darkening. Half of the crops receive one to four real filaments pasted from other training frames [5]. The loss is binary cross-entropy on the soft target, plus 0.5 soft Dice, plus 0.2 of a centre-line term (cross-entropy on the auxiliary head and skeleton recall [11] on the main one) that penalises a broken filament. I train with AdamW, learning rate $3 \cdot 10^{-4}$ (half of that in the encoder), 5% warm-up and a cosine decay, for 150 epochs of 80 steps, and evaluate an exponential moving average of the weights (decay 0.999). One fold takes about 1.9 hours on an RTX PRO 6000. The submitted model is trained on all 707 frames with the same recipe.

Candidates. At test time the whole frame goes through the network in four flipped and rotated views, the map is set to zero outside the solar disk and thresholded at 0.8, and every 8-connected component of at least 30 pixels is a candidate. The masks are disjoint by construction, which the leaderboard requires.

Keep rule. Adding a candidate to a frame with n records adds its matched IoU to the numerator of PQ for every record it matches and exactly $n/2$ to the denominator. Keeping it therefore raises PQ exactly when $y = \frac{1}{n}\sum_r \mathrm{IoU}_r\,[\mathrm{IoU}_r > 0.5]$ is larger than PQ/2. I estimate y with two regressors and keep a candidate when the mean of the two estimates exceeds $\tau = 0.25$.

The *feature scorer* is the gradient-boosted regressor, on twenty numbers per candidate: area and shape, statistics of the probability inside it, its darkness against a surrounding ring, its distance from disk centre and the number of candidates in the frame.

The *patch scorer* is the ResNet-18, and it looks at the candidate. Its input is a 128 px patch with six channels: image, probability map and candidate mask in a window of 1.5 times the bounding box, and the same three in a window of four times the box for context. It is trained with cross-entropy on the soft target y, with flips, rotations and a brightness jitter; three seeds and the eight symmetries are averaged. The patch is scale-normalised and so loses the absolute size of the candidate, which the feature scorer keeps; this is one reason the mean of the two is better than either. Both scorers are fitted on candidates cut from out-of-fold maps. The kept masks are written as run-length codes. Figure 2 shows examples.

3 EVALUATION

Strategy. Every number is out-of-fold on the 707 training frames: the network that drew a mask never saw the frame, and each scorer was fitted on four folds and applied to the fifth. PQ is pooled over all 1,154 annotator records with my implementation of the metric, which I tested against the host's self-evaluation notebook. Intervals are 90% bootstrap intervals over frames (2,000

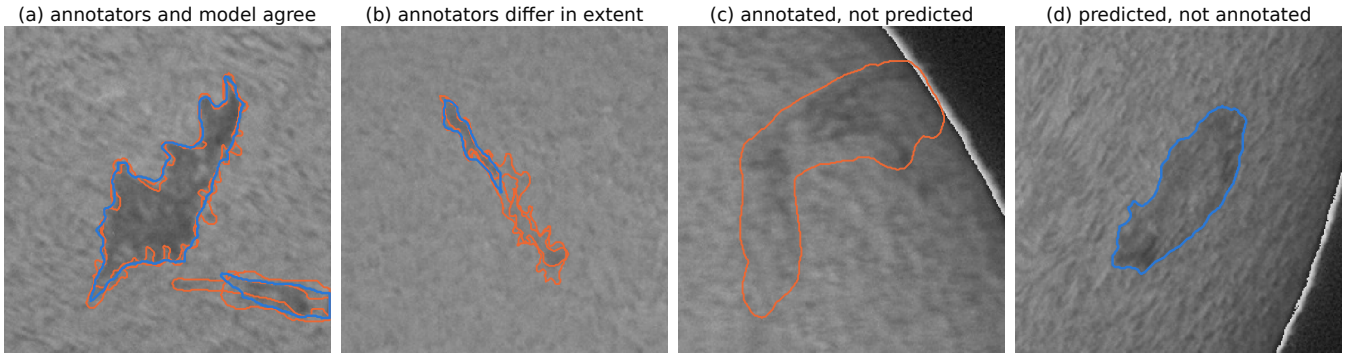


Figure 2: Out-of-fold close-ups, 288×288 px. Orange: every annotator of the frame; blue: kept prediction. (a) A filament with barbs on which the annotators and the model agree. (b) Two annotators draw the same filament with different extent, and the model follows the shorter outline. (c) A broad, faint filament at the limb that the only annotator of this frame drew and the model missed. (d) A dark feature near the limb that the model keeps and no annotator drew; the metric counts it as a false positive.

Table 2: Out-of-fold PQ on all 707 frames, built up step by step.

pipeline	PQ
previous version: 3 U-Nets, 50 epochs, feature scorer	0.4455
one ConvNeXt-T U-Net, 150 epochs, components only	0.402
+ feature scorer	0.4547
+ patch scorer instead	0.4547
+ mean of both scorers (submitted)	**0.4586**
90% bootstrap interval	0.451–0.467
lowest and highest fold	0.447–0.473
with perfect keep decisions (upper bound)	0.519

resamples). The threshold 0.8 and τ are the only two numbers tuned on the pooled out-of-fold score, so the estimate is slightly optimistic. Nothing was tuned on the leaderboard.

Panoptic Quality. Table 2 shows the path to the submitted pipeline. The final result is PQ 0.4586 with segmentation quality 0.683 and recognition quality 0.671 (5,143 matches, 1,981 unmatched predictions, 3,056 unmatched annotations). The model agrees with an annotator better than a second annotator does (0.34). Teams that published both numbers lose about 0.06 between out-of-fold and the public board, and my earlier versions did the same, so I expect about 0.39 to 0.40 there.

IoU, Dice and one-to-many relations. Figure 3 gives the distributions behind the score. The matched filaments have a mean IoU of 0.683 (median 0.691) and a mean Dice of 0.809 (median 0.818). 855 annotated filaments (10%) are near misses with a best IoU between 0.3 and 0.5, and 1,849 (23%) have no predicted mask on them at all: these are mostly faint filaments that one annotator drew, and keeping more candidates to reach them costs more in false positives than it returns. Fragmentation and merging are rare. Only 187 annotated filaments (2.3%) are covered by two or more predicted masks, and 201 predicted masks (2.8%) lie over two or more filaments of the same annotator; 783 predicted masks (11%) overlap no annotation of the record they are scored against.

Table 3: Variants I rejected. PQ through the complete decoder; fold 0 (141 frames) unless marked * (all folds). "Without" is the submitted 150-epoch network; the tree row uses the maps of my previous version and the annotator row the 411 frames with one annotator. On one fold, differences below 0.005 are noise.

change	without	with
Jaccard metric loss added	0.446	0.440
ConvNeXt-S encoder	0.446	0.440
input magnified 2×	0.446	0.437
stochastic depth 0.2	0.446	0.441
three 150-epoch networks averaged	0.446	0.447
six 50-epoch networks averaged	0.446	0.436
labels copied to GONG frames ±2 min	0.448	0.441
extent chosen from a component tree*	0.446	0.437
annotator known to the scorer*	0.458	0.461
crop refiner after the keep rule*	0.4586	0.4600

What did not help. Table 3 lists what I tested and rejected. Two lessons were worth more than any single row. The PQ of the raw components is a poor guide: several changes raised it by up to 0.01 and changed nothing once the scorers had made their decisions, so I compared every variant through the complete decoder. And averaging a strong map with weaker ones lowers the score, which is why the submission has one network where my previous version had three. Longer training was the only change on the network side that survived (+0.009), and the patch scorer the only one in the decoder (+0.004 here, +0.007 on the maps of the previous version).

Three rows deserve a sentence. For the *neighbour frames* I downloaded 1,818 GONG frames taken within 2.5 minutes of a training frame at the same site, which the organisers allow for training,

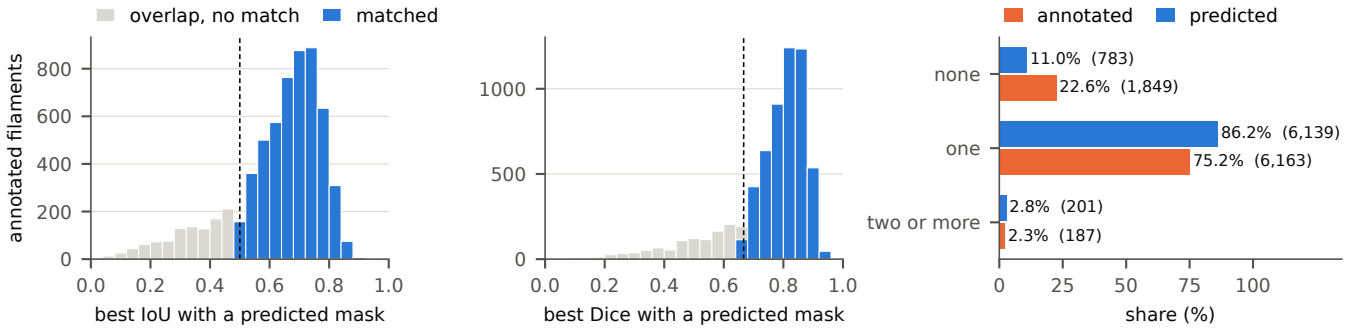


Figure 3: Out-of-fold predictions against all 1,154 annotator records. Left and middle: best IoU and best Dice of each annotated filament with a predicted mask (the dashed line is the match threshold; the 1,849 filaments without any overlap are not drawn). Right: how many masks of the other kind overlap an annotated filament and a predicted mask. "Two or more" is fragmentation for the annotated filaments and merging for the predicted masks.

checked their alignment (median shift 0.35 px) and showed each training frame either as itself or as one of its neighbours. The model was 0.007 worse. The neighbours add new pictures of the same filaments but no new opinion about them. The *component tree* offers, for each filament, its outline at eight thresholds. A perfect choice among them would reach 0.57, but every learned chooser fell back below the single threshold: which extent an annotator draws cannot be read from the image. The *crop refiner*, a second network that redraws each kept filament in its own window, moved the mean IoU of the matches from 0.683 to 0.684.

Strengths. On the three difficulties named by the organisers: *fine structure*, the frame is never downsampled and the masks follow barbs and narrow necks where the annotators draw them (Figure 2a); *noise*, the augmentation imitates seeing, thin cloud and limb darkening, and the scorers remove most of the dark specks that survive the threshold (Figure 1c); *continuity*, the centre-line term keeps thin filaments in one piece, and 2.3% of the annotated filaments end up fragmented. The pipeline is also cheap. It runs one network on the full frame and three ResNet-18 scorers on about ten small patches: the 180 test frames take 63 s on an RTX PRO 6000, that is 0.35 s per frame including image decoding and writing the CSV (my previous three-network version needed 0.57 s). The weights used at test time are 64 MB for the network and 69 MB for the scorers.

Weaknesses and what could improve them. The model is weak where the annotators are least consistent: faint, broad filaments near the limb (Figure 2c) and small filaments that a single annotator marked. It also draws features that look like filaments and were not annotated (Figure 2d), which the metric penalises. The gap between 0.459 and the 0.519 of perfect keep decisions is what a better scorer could still win, and each of my attempts at it brought thousandths. I see two ways forward, neither inside this competition's data: more frames drawn by three annotators, so that the target is a consensus and not one person's choice, and a scorer that sees the same region a few hours later, since a real filament persists and a dark speck does not.

Data and rules. I trained only on the images and polygons of the Kaggle training set, with encoders pretrained on ImageNet. The 1,818 unlabelled GONG frames were used in one rejected experiment and are not part of the submission; none of them is a test frame or lies within an hour of one. I did not download the full MAGFiLO release, which contains annotations of the test frames, and I used no other ground truth. At test time the pipeline sees only the supplied frame.

Code availability. The code, the weights of all six networks and of both scorers, the frozen folds and a notebook that reproduces the submission from the test images are at https://github.com/Muhammad-Elias-Muhammadi/Solar-Filament-Segmentation. Running `scripts/predict.py` there writes the submitted file byte for byte.

4 ACKNOWLEDGMENT

The organization of the Filament Segmentation Challenge 2026 is partially supported by the U.S. National Science Foundation (NSF) under Grant No. 2209912 and 2433781, directorate for Computer and Information Science and Engineering (CSE), and Office of Advanced Cyberinfrastructure (OAC), and Grant No. 2511630, AST Division Of Astronomical Sciences and MPS Directorate for Mathematical and Physical Sciences.

The Filament Segmentation Challenge 2026 is sponsored by the U.S. National Science Foundation (NSF) National Solar Observatory (NSO).

This work utilizes GONG data obtained by the NSO Integrated Synoptic Program, managed by the National Solar Observatory, which is operated by the Association of Universities for Research in Astronomy (AURA), Inc. under a cooperative agreement with the National Science Foundation and with contribution from the National Oceanic and Atmospheric Administration. The GONG network of instruments is hosted by the Big Bear Solar Observatory, High Altitude Observatory, Learmonth Solar Observatory, Udaipur Solar Observatory, Instituto de Astrofísica de Canarias, and Cerro Tololo Interamerican Observatory.

REFERENCES

[1] Azim Ahmadzadeh, Rohan Adhyapak, Kartik Chaurasiya, Laxmi Alekhya Nagubandi, V. Aparna, Petrus C. Martens, Alexei Pevtsov, Luca Bertello, Alexander Pevtsov, Naomi Douglas, Samuel McDonald, Apaar Bawa, Eugene Kang, Riley Wu, Dustin J. Kempton, Aya Abdelkarem, Patrick M. Copeland, and Sri Harsha Seelamneni. 2024. A dataset of manually annotated filaments from H-alpha observations. In *Nature Scientific Data (Nature Scientific Data, Vol. 11)*. Springer, Article 1031, 1031 pages. https://doi.org/10.1038/s41597-024-03876-y

[2] Azim Ahmadzadeh, Dustin J. Kempton, Qin Li, and Alexei A. Pevtsov. 2026. Solar Filament Segmentation Challenge 2026. https://kaggle.com/competitions/filament-segmentation-2026. Kaggle.

[3] Azim Ahmadzadeh, Sushant S. Mahajan, Dustin J. Kempton, Rafal A. Angryk, and Shihao Ji. 2019. Toward Filament Segmentation Using Deep Neural Networks. In *2019 IEEE International Conference on Big Data (Big Data)*. 4932–4941. https://doi.org/10.1109/BigData47090.2019.9006340

[4] Andrea Diercke, Robert Jarolim, Christoph Kuckein, Sergio J. González Manrique, Marco Ziener, Astrid M. Veronig, Carsten Denker, Werner Pötzi, Tatiana Podladchikova, and Alexei A. Pevtsov. 2024. A universal method for solar filament detection from Hα observations using semi-supervised deep learning. *Astronomy & Astrophysics* 686 (2024), A213. https://doi.org/10.1051/0004-6361/202348314

[5] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Rui Qian, Tsung-Yi Lin, Ekin D. Cubuk, Quoc V. Le, and Barret Zoph. 2021. Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[6] Melody Guan, Varun Gulshan, Andrew Dai, and Geoffrey Hinton. 2018. Who Said What: Modeling Individual Labelers Improves Classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32. https://doi.org/10.1609/aaai.v32i1.11756

[7] Xulong Guo, Yunfei Yang, Song Feng, Xianyong Bai, Bo Liang, and Wei Dai. 2022. Solar-Filament Detection and Classification Based on Deep Learning. *Solar Physics* 297 (2022), 104. https://doi.org/10.1007/s11207-022-02019-z

[8] J. W. Harvey et al. 2011. Full-disk Solar H-alpha Images From GONG. In *AAS/Solar Physics Division Meeting 42*.

[9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[10] Pavel Iakubovskii. 2019. Segmentation Models PyTorch. https://github.com/qubvel-org/segmentation_models.pytorch.

[11] Yannick Kirchhoff, Maximilian R. Rokuss, Saikat Roy, et al. 2024. Skeleton Recall Loss for Connectivity Conserving and Resource Efficient Segmentation of Thin Tubular Structures. In *European Conference on Computer Vision (ECCV)*.

[12] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. 2019. Panoptic Segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. https://doi.org/10.1109/CVPR.2019.00963

[13] Dan Liu, Wei Song, Ganghua Lin, and Haimin Wang. 2021. Solar Filament Segmentation Based on Improved U-Nets. *Solar Physics* 296 (2021), 176. https://doi.org/10.1007/s11207-021-01920-3

[14] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. 2022. A ConvNet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[15] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[16] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.

[17] Ross Wightman. 2019. PyTorch Image Models. https://github.com/huggingface/pytorch-image-models.

[18] Gaofei Zhu, Ganghua Lin, Dongguang Wang, Suo Liu, and Xiao Yang. 2019. Solar Filament Recognition Based on Deep Learning. *Solar Physics* 294 (2019), 117. https://doi.org/10.1007/s11207-019-1517-4